F20BD/F21BD – Big Data Management

WEEK 5: CAP THEOREM & NoSQL

Complete Lecture Notes – Every Slide Covered

Slides Covered:

1. Relational Database Technology & ACID
2. RDBMS Limitations & Internet Challenges
3. Distributed Databases: Fragmentation, Sharding, Replication
4. **Dimensions & Taxonomy of Distributed DBs**
5. **Challenges of Distributed Systems**
6. CAP Theorem (Properties, History, CA/CP/AP)
7. **Critics of CAP & Replication Strategies**
8. NoSQL History, Characteristics, BASE Principles
9. Four NoSQL Data Models & CAP Mapping
10. **PACELC Theorem**
11. **NoSQL Aims, Characteristics & RDB vs NoSQL**

• Items in red were MISSING from previous notes

Heriot-Watt University – AY 2025-26

• **UPDATED VERSION:** Every slide from the Week 5 lecture is now covered. All gaps from the previous notes have been filled. Written for **zero prior knowledge**.

Last updated: February 25, 2026

Contents

1 Relational Database Technology

• Definition:

Relational Database (RDBMS) A database system that stores data in structured **tables** (rows and columns) with predefined schemas. Uses SQL (Structured Query Language) for queries. **Invented in the 1970s**. Well-founded theoretically on **relational algebra**.

Key strength: Provides **high consistency** and is excellent for **centralised storage** (one powerful server).
Examples: Oracle, MySQL, PostgreSQL, Microsoft SQL Server, DB2, SQLite.

• EXAM TIP:

RDBMS Added Capabilities – Memorise This List The lecture lists specific added capabilities of RDBMS. Know them:

Capability	What It Means
BLOB storage	Store large binary objects (images, videos)
Document store	Store and query documents within the DB
Text search	Full-text search capabilities
In-database code execution	Run code (stored procedures) inside the DB
Data warehousing (star schema)	Analytical structures for business intelligence
GIS extensions	Geographic/spatial data support

2 ACID Properties in RDBMS

ACID is a set of four guarantees that traditional relational databases follow to ensure data is always reliable:

Letter	Property	Lecture Definition	Simple Explanation
A	Atomicity	The entire transaction takes place at once or doesn't happen at all.	All-or-nothing. Like a bank transfer: BOTH debit and credit happen, or NEITHER does.
C	Consistency	The data must be consistent before and after the transaction.	Database goes from one valid state to another. No broken rules ever.
I	Isolation	Multiple transactions occur independently without interference.	Concurrent transactions don't mess with each other. Each appears to run alone.
D	Durability	The changes of a successful transaction must persist (even if the system fails).	Once committed, it's permanent – survives crashes, power loss, etc.

• Real-World Analogy:

ACID at a Bank ATM **Atomicity:** You transfer £100 from Account A to B. Either BOTH the deduction from A and addition to B happen, or NEITHER does.
Consistency: The total money across all accounts always adds up correctly – before and after every transaction.
Isolation: You and another person both use ATMs at the same time. Your transactions don't interfere with each other.
Durability: Once you see "Transaction Complete," even if the bank's power goes out one second later, your transfer is permanent.

3 RDB Technology and the Internet

• Key Concept:

The Internet Changed Everything When databases went online (intensive internet access), the requirements shifted dramatically:

- **Availability** – the system must always respond
- **Scalability** – handle growing numbers of users
- **Speed** – fast response times

Critical insight from the lecture: "Consistency cannot be ensured at all times" when serving internet-scale traffic. This is the fundamental tension that leads to the CAP theorem and the need for distributed databases.

4 Limitations of Relational Databases

• EXAM TIP:

Three RDBMS Limitations – From the Slides The lecture lists exactly three limitations. Memorise them word-for-word:

1. **Low response time with massive amounts of data** – fast querying becomes difficult when tables have billions of rows
2. **Expensive Scale-up** – adding memory, CPU to one machine costs a fortune and has physical limits
3. **Limited analytical capabilities** – need special OLAP-like schemas (star schema, snowflake schema) for analytics

These limitations led to the **appearance of distributed databases (DDBMS)**.

• Definition:

Distributed Database (DDBMS) A system where:

- Data is stored in **different nodes** (computers)
- Nodes are **interconnected by a computer network**
- Each node IS a database
- **DDBMS software** manages the distributed database (handles access and **transparent distribution**)

"Transparent" means the user doesn't know or care which node stores their data.

5 Fragmentation, Replication, and Sharding

5.1 Fragmentation (Partitioning) and Replication

• Key Concept:

Distribution is Transparent The lecture emphasises: **Users are unaware of fragments and their replication.** Queries are specified on the **original relations (schema)**, NOT on individual fragments. The DDBMS handles routing.

• Example:

How Fragments are Distributed Across Sites From the lecture diagram: Table T is broken into 4 fragments (T1-T4), each replicated across sites:

- **Site A:** Copy1 of T1, Copy1 of T3
- **Site B:** Copy2 of T1, Copy1 of T2
- **Site C:** Copy3 of T1, Copy2 of T3

Notice T1 has THREE copies (high replication for important data), while T4 has only one copy somewhere.

5.2 Horizontal vs Vertical Partitioning

	Horizontal Partitioning	Vertical Partitioning
What it splits	**Rows** – different subsets of rows on different servers	**Columns** – different groups of columns on different servers
Each fragment has	All columns, but only some rows	The primary key PLUS some columns
Example	Users 1-1000 on Server A, Users 1001-2000 on Server B	(ID, Name) on Server A; (ID, Avatar, Bio) on Server B
Key rule	Every row appears in exactly one fragment	The **key (ID) must be in EVERY fragment** so they can be reassembled

• Real-World Analogy:

Horizontal = Cutting a Pizza into Slices Imagine a table is a pizza. **Horizontal partitioning** cuts it into slices (each slice has all toppings but only part of the pizza). **Vertical partitioning** separates the toppings – cheese on one plate, pepperoni on another, but each plate has a label (the primary key) so you know which pizza they belong to.

5.3 Database Sharding

• Definition:

Database Sharding Sharding is **another way of naming partitioning**, but specifically:

- Table is partitioned into **shards**
- Shards are **necessarily stored on different computers**
- Queries are **run on separate shards**

• EXAM TIP:

Facebook Sharding Fact **In 2011, Facebook split its MySQL database into 4,000 shards** to handle the site's massive data volume. This specific fact appears on the lecture slide – expect it on the test.

6 Dimensions of Distributed Database Systems

• CLASS TEST ALERT:

This Section Was MISSING From Previous Notes! The entire "Dimensions" slide was not covered before. These three dimensions (Distribution, Heterogeneity, Autonomy) and their levels are directly testable. Learn the codes (D0, D1, D2, H0, H1, A0, A1, A2).

Distributed database systems are characterised along **three dimensions**:

6.1 Dimension 1: Distribution (How is data shared?)

Code	Level	Meaning
D0	No distribution	Everything on one machine. Not distributed at all.
D1	Client-server	Clients send requests to a central server that holds the data. The server is the authority.
D2	Peer-to-peer	All nodes are equal – any node can accept reads and writes. No central authority. Most distributed mode.

• Real-World Analogy:

Distribution Levels **D0** = One person has the only copy of a book (centralised).
D1 = A library (server) holds all books; readers (clients) must go there to read.
D2 = Everyone has copies of every book and can lend to anyone (peer-to-peer, like Cassandra).

6.2 Dimension 2: Heterogeneity (Same or different tech?)

Code	Level	Meaning
H0	Homogeneous	All nodes use the **same** database software/technology.
H1	Heterogeneous	Different nodes use **different** database technologies (e.g., one node runs MySQL, another runs MongoDB).

6.3 Dimension 3: Autonomy (How independently do nodes operate?)

Code	Level	Meaning
A0	Tight integration	A **single coordinator** controls everything. Nodes have no independence.
A1	Semi-autonomous	Nodes have some independence but implement **sharing** capabilities.
A2	Fully independent	Nodes are completely independent. Sharing is done by **another software layer**, not the nodes themselves.

7 Taxonomy of Distributed Database Systems

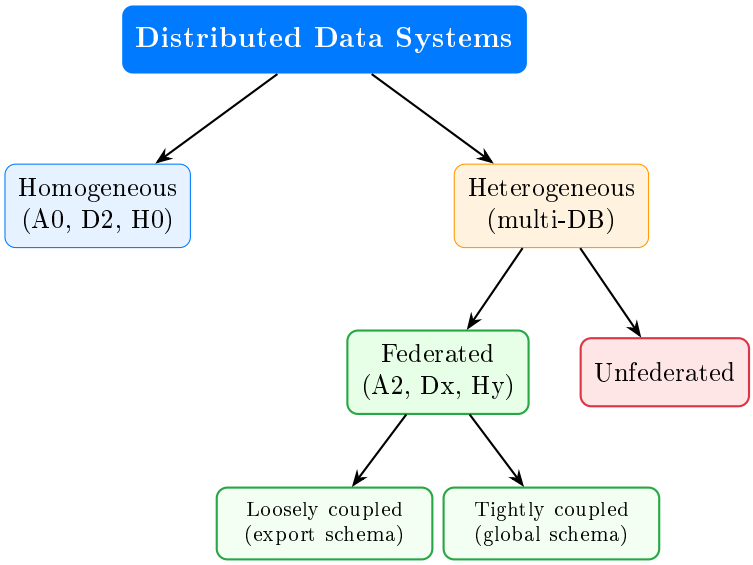
• CLASS TEST ALERT:

This Section Was MISSING From Previous Notes! The taxonomy tree (homogeneous vs heterogeneous, federated vs unfederated, loosely vs tightly coupled) and the definitions of DBMS vs multi-DB global schemas are testable content.

• Definition:

Key Definitions from the Slide **Distributed database:** A logically integrated collection of shared data, physically distributed across network nodes.
In a DBMS: The global schema is the **union** of local schemas of all databases (everything is visible globally).
In a multi-DB: The global schema is a **subset** of the union of all local databases (only some data is shared globally).

7.1 The Taxonomy Tree



Type	Codes	Description
Homogeneous	A0, D2, H0	All nodes use the SAME tech (H0), peer-to-peer (D2), tightly integrated under one coordinator (A0).
Federated	A2, Dx, Hy	Fully independent nodes (A2) that interoperate. Can be **loosely coupled** (communicate via export schemas) or **tightly coupled** (share a global schema).
Unfederated	varies	Heterogeneous system with no coordinated interoperation.

• Real-World Analogy:

Federated vs Homogeneous **Homogeneous** = A chain of McDonald's restaurants. All use the same equipment, same recipes, same systems. One central HQ coordinates everything.
Federated (loosely coupled) = Different restaurants (McDonald's, KFC, Burger King) agree to accept each other's gift cards. Each runs independently but they cooperate through a shared

interface (export schema).
Federated (tightly coupled) = Same different restaurants but with a shared ordering system (global schema) that knows about all menus.

8 Challenges of Distributed Database Systems

• CLASS TEST ALERT:

This Section Was MISSING From Previous Notes! The specific three requirements and four challenges from this slide are directly testable.

8.1 Three Requirements for Internet-Scale Applications

Large-scale internet applications require scalable storage and support of concurrency. Specifically:

1. **Increasable data capacity** – storage must grow as data grows
2. **Growing read/write throughput** – handle more operations per second over time
3. **Handle highly concurrent access** – thousands/millions of users at the same time

8.2 Four Key Challenges

Horizontal scaling is required for throughput and capacity increases, **however**:

1. Database must **maintain data consistency** while read/write operations are conducted
2. In case of **node failure**, the overall system must **maintain availability**
3. Operations need **low latency** (fast responses)
4. All of this must work simultaneously – which is *extremely* difficult

• Key Concept:

Why This Matters These challenges are exactly what the CAP theorem addresses. You **cannot** solve all these challenges simultaneously – you must make trade-offs. That's the entire point of CAP.

9 Properties of Distributed Systems (C, A, P)

Before understanding the CAP theorem, you must know the **exact lecture definitions** of each property:

Property	Lecture Definition (Memorise!)	Simple Explanation
Consistency	The system assures that operations are atomic, and changes are disseminated simultaneously to all nodes, with same results. **A read is guaranteed to return the most recent write.**	Update your photo on Node 1 → reading from ANY node shows the new photo immediately.
Availability	The system must answer every request, even in case of failures. A **non-failing node** returns a reasonable response within a reasonable time, **without guarantee of being the latest write.**	You ALWAYS get a response. But it might be slightly outdated.
Partition Tolerance	**Partition = any arbitrary split between nodes resulting in message loss.** The system must be resilient to message losses between nodes. Consistency mechanisms must cope with message losses. Availability requires any partition to be able to answer a request.	Even if the network cable between data centres gets cut, the system keeps working.

• EXAM TIP:

Key Phrases to Memorise **Consistency:** "A read is guaranteed to return the most recent write" **Availability:** "A non-failing node returns a reasonable response within a reasonable time, without guarantee of being the latest write"
Partition: "Any arbitrary split between nodes resulting in message loss in between"

10 The CAP Theorem

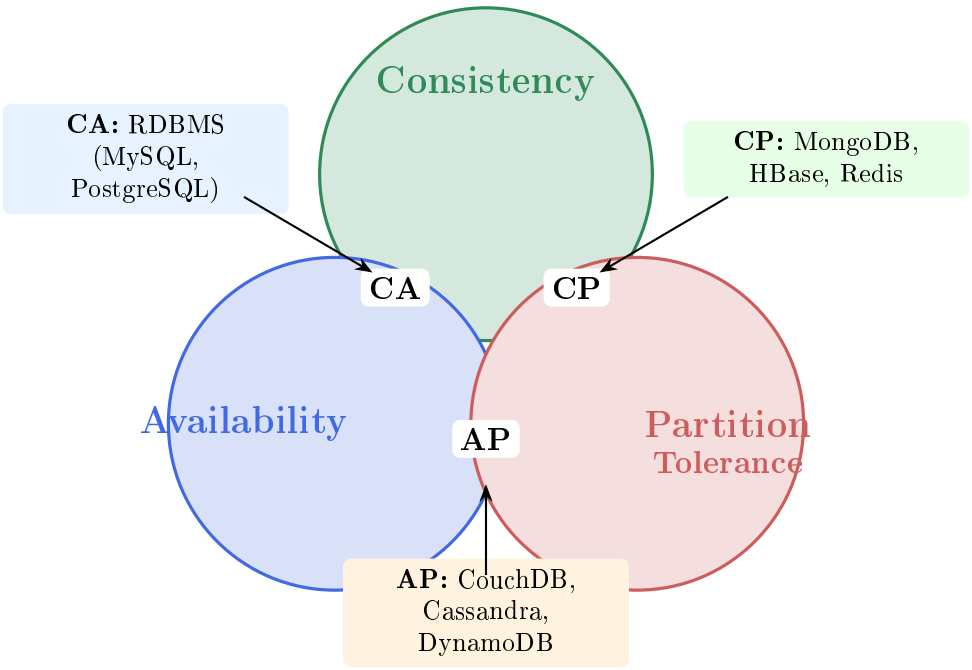
• Definition:

The CAP Theorem A distributed system can guarantee **at most TWO** of the three properties (Consistency, Availability, Partition Tolerance) **at the same time**. You cannot have all three. The possible combinations are: **CA, CP, or AP**.

History:

- **2000:** Eric Brewer proposed this as a **Conjecture** (an educated guess)
- **2002:** Gilbert and Lynch **demonstrated it as a Theorem** (mathematically proven)

Important: It started as a conjecture and was later proven. The lecture uses both terms deliberately.



10.1 CA Systems – Consistency + Availability

• Key Concept:

CA – Consistent and Available, but No Partition Tolerance **What you get:** Consistent AND available services.
What you sacrifice: Cannot tolerate partitions. **In case of partition, systems become inconsistent** (conflicting replicas).
Who uses it: Traditional **RDBMS** (MySQL, PostgreSQL, Oracle).

Mechanisms used:

- **Replication** is used to achieve high availability
- **Transaction protocols (e.g. two-phase commit)** ensure consistency
- Partitions lead to **conflicting replicas**

• EXAM TIP:

Two-Phase Commit – Now Explicitly Covered **Two-phase commit** is the specific mechanism CA systems use for consistency. It works in two phases: (1) **Prepare phase:** coordinator asks all nodes "can you commit?" (2) **Commit phase:** if all say yes, coordinator says "commit"; if any says no, coordinator says "rollback." This ensures all nodes agree.

When a CA system faces a partition, it has two recovery options:

1. **Favour Consistency → CP:** Require consensus protocol. No answer if no consensus. Sacrifice availability.
2. **Favour Availability → AP:** Returned data is not guaranteed to be the latest update.

10.2 CP Systems – Consistency + Partition Tolerance

• Key Concept:

CP – Consistent Even During Partitions **What you get:** Strongly consistent service, even when the network splits.
What you sacrifice: Availability – the system might refuse to answer.
How it works: Nodes must **agree** to ensure consistency. Uses **quorum and majority decision algorithms** (e.g. **Paxos protocol**).
During a partition: Consensus is delayed, therefore affecting availability (**no answer possible**).
Who uses it: MongoDB, HBase, Redis.

Philosophy: "I'd rather give you **nothing** than give you something **wrong**."

10.3 AP Systems – Availability + Partition Tolerance

• Key Concept:

AP – Always Available, Even During Partitions **What you get:** Available service, even when the network splits.
What you sacrifice: Consistency – you might get outdated data.
During a partition: Unreachable nodes hold **temporarily inconsistent data**.
Recovery mechanism: Eventual consistency – the system converges to full consistency over time.
Who uses it: CouchDB, Cassandra, DynamoDB, Riak.

Philosophy: "I'll **always** give you an answer, but it might be **slightly outdated**."

• Real-World Analogy:

The Pizza Delivery CAP Analogy Three promises: (1) **C:** correct pizza, (2) **A:** always deliver, (3) **P:** deliver even when roads blocked.
When a road is blocked (partition):

- **CP:** Wait until the road clears, then deliver the correct pizza (correct but slow/unavailable)
- **AP:** Send whatever pizza is available from the nearest location (always delivers but might be wrong)

You can't guarantee all three when roads are blocked!

10.4 The Key Question: RDBMS → NoSQL

• Warning:

Direct Quote from the Lecture The lecture poses this question: "Since RDBMS have CA model, therefore cannot provide availability or consistency in the presence of partition – **which database systems can be designed according to CP or AP models?**"
Answer: NoSQL database systems!
This is the bridge between traditional databases and NoSQL. RDBMS = CA (no partition tolerance). When you need CP or AP for distributed systems, you need NoSQL.

11 NoSQL Movement: History

Year	Event
1998	**Carlo Strozzi** uses "noSQL" to name a **relational** DB queried without SQL
2000	Neo4j (graph database) created
2004	Google BigTable (wide-column) – research paper
2005	CouchDB created
2007	Amazon DynamoDB – research paper published
2008	Cassandra **open-sourced by Facebook**
2009	"NoSQL" used as arbitrary name of conference hosted by **Rackspace**

• EXAM TIP:

Three Tricky Facts

1. Strozzi's 1998 "noSQL" was ironically a **relational** database – just one queried without SQL!
2. The movement started as "**no more SQL**" (anti-SQL) then softened to "**Not Only SQL**"
3. Cassandra was open-sourced by **Facebook** (not Google!) in 2008

12 Characteristics of NoSQL Databases

Characteristic	Details from Lecture
Do NOT respect ACID	Consistency cannot be achieved while targeting high availability. Substituted by **eventual consistency** (in a parallel environment).
No standard declarative query language	Most use variations of SQL. Examples: **Cassandra CQL**, **Couchbase N1QL** (SQL-like), Cosmos DB, Elasticsearch SQL, MongoDB query language.
No predefined schema	Rely on **dynamic schema** – structure can change without altering the database definition first.
Horizontal scaling	Built to **scale out** (add more machines) vs RDBMS which scale up (bigger machine).

• EXAM TIP:

Specific Query Languages – Now Covered The lecture names specific NoSQL query languages. Know these examples:

- **Cassandra CQL** (Cassandra Query Language)
- **Couchbase N1QL** (pronounced "nickel" – SQL-like)
- Cosmos DB, Elasticsearch SQL, MongoDB (MQL)

The key point: there is NO single standard like SQL – every system has its own language/API.

13 BASE Principles

• Definition:

BASE – The NoSQL Alternative to ACID **B**asic **A**vailability – Highly distributed DBMS, with replication, to provide availability **even with failures**.
Soft State – Consistency is the problem of the **developer**, not the DBMS. The system state may change over time without input as replicas synchronise.
Eventual Consistency – Database should converge to a consistent state in the future. **Operations can be executed without waiting for prior consistency.**

	ACID (Relational/SQL)	BASE (NoSQL)
Consistency	**Strong** – always correct	**Eventual** – correct later
Availability	Less available	**Availability first**
Who handles consistency?	The DBMS automatically	The **developer/programmer**
Approach	Conservative (pessimistic)	Aggressive (optimistic)
Scaling	Vertical (scale up)	Horizontal (scale out)

14 Four NoSQL Data Models

• EXAM TIP:
Know All Four + Examples Be able to name all four models, describe how each stores data, and give example databases for each. This is practically guaranteed on any test covering Week 5.

14.1 1. Key-Value Store

Feature	Details
Data structure	Hashtable (dictionary or map) – each entry has a **key** and a **value**
Values are "opaque"	The database **CANNOT see inside** the value. It can only store and retrieve by exact key.
Schema	Schema-less
Storage	Mostly **in-memory** (very fast)
Examples	**Redis**, **Riak KV**, Memcached, SimpleDB, Oracle NoSQL, **DynamoDB***

**DynamoDB and Cosmos DB are listed as both key-value AND document stores.*

14.2 2. Document Store

Feature	Details
Extends	Key-value model to **key-document** lookup
Data format	Structured data (set of key-value pairs) stored in **JSON or XML** – hence "document"
Key difference	Database **HAS access to internal structure** → can query individual fields
Indexing	Indexes used for fast query; documents indexed by unique key
Search	Includes **search capabilities** (search patterns)
Examples	**MongoDB** (BSON/JSON), **CouchDB**, DynamoDB, Couchbase, Elasticsearch

• Warning:
Key-Value vs Document – The Critical Difference **Key-Value:** Values are **OPAQUE** – the DB can't look inside. It's a black box.
Document: Values are **STRUCTURED** (JSON/XML) – the DB CAN look inside and query individual fields.
This is the most commonly tested distinction between these two models!

14.3 3. Wide-Column Store

Feature	Details
Also called	Column family databases
Structure	"Flexible" tables with rows and columns. **Sparse data matrix.**
Key feature	Names and formats of columns can **vary from row to row**. Not all columns have values.
Vertical sharding	**Column families** (sets of related columns) stored on **separate computers**
Horizontal sharding	Within a column family, **rows** are stored on **different computers**
Examples	**Cassandra**, **HBase**, Google BigTable, Accumulo, HyperTable

• Example:

Wide-Column vs Relational – Sparse Data **Relational table** (every row MUST have all columns):

Name	Email	Phone	Fax
Alice	alice@x.com	123-456	*NULL*
Bob	bob@y.com	*NULL*	*NULL*

Wide-column store (each row stores only what it HAS):

Row Key	Columns (only what exists)
Alice	email:alice@x.com, phone:123-456
Bob	email:bob@y.com

No wasted NULLs! Much more space-efficient for sparse data.

14.4 4. Graph Database

Feature	Details
Focus	**Relationships** between entities
Structure	**Nodes** = entities (people, products). **Edges** = relations/properties between them.
Use case	**Widely used by social networks** (friends-of-friends, recommendations)
Examples	**Neo4j**, Titan, InfiniteGraph, Giraph

15 NoSQL and CAP Properties

• EXAM TIP:

Which Database = Which CAP Model? This table is a MUST-MEMORISE for the test. Know which databases go in which category:

Database	Type	CAP	Key Feature
MySQL, PostgreSQL, Oracle	Relational	**CA**	Strong consistency, two-phase commit
MongoDB	Document	**CP**	Consistent even with partitions
HBase	Wide-column	**CP**	Consistent + partition tolerant
Redis	Key-value	**CP**	Fast, in-memory, consistent
CouchDB	Document	**AP**	Always available
Cassandra	Wide-column	**AP**	Highly available, eventual consistency
DynamoDB	Key-value/Doc	**AP**	AWS managed, highly available
Riak	Key-value	**AP**	Distributed, always available

16 Critics of CAP Theorem – Replication Strategies

• CLASS TEST ALERT:

This Entire Section Was MISSING From Previous Notes! The "Critics of CAP" slide contains detailed replication strategies (synchronous vs asynchronous, pre-processing layers) that are directly testable. This is now fully covered.

16.1 The Core Criticism

The CAP theorem's trade-off between Consistency and Availability **only applies during partitions**. But the lecture points out:

• Warning:

CAP Is Incomplete **"Even in the absence of partition (normal operations), one has to choose between Latency and Consistency."**
High availability requires replication. When you replicate data, you must choose how to propagate updates – and EVERY choice involves a trade-off.

16.2 Three Replication Strategies

The lecture describes three approaches to sending data updates to replicas:

Strategy	Variant	Result	Trade-off
1. Send to ALL replicas simultaneously	Without pre-processing layer	**Latency** (slow)	Must wait for all to acknowledge
	With pre-processing layer	**Consistency** affected	Pre-processing adds delay to ensure order
2. Send to agreed-upon location first	Synchronous	**Consistency**	All nodes update together (slow but correct)
	Asynchronous	**Latency** (fast)	Faster but nodes may be out of sync (e.g. PNUTS)
3. Send to arbitrary location (Dynamo, Cassandra, Riak)	Synchronous	**Consistency**	Wait for propagation
	Asynchronous	**Latency** (fast)	Faster responses, temporary inconsistency

• Key Concept:

The Pattern In EVERY strategy: **Synchronous replication** → gives **Consistency** (but slower).
Asynchronous replication → gives **low Latency** (but risks inconsistency).
This is the Latency vs Consistency trade-off that PACELC captures in its "Else" clause (see next section).

17 PACELC Theorem

• Definition:

PACELC Theorem An extension of CAP that captures what happens during **normal operations** too:

- **If there is a Partition (P):** trade-off between **Availability (A)** and **Consistency (C)**
- **Else (E), during normal operation:** trade-off between **Latency (L)** and **Consistency (C)**

Partition → **A**vailability vs **C**onsistency / **E**lse → **L**atency vs **C**onsistency

Database	On Partition	Else (Normal)	PACELC	Meaning
Cassandra	Availability (PA)	Latency (EL)	**PA/EL**	Always fast, always available
MongoDB	Consistency (PC)	Consistency (EC)	**PC/EC**	Always correct, even if slower
DynamoDB	Availability (PA)	Latency (EL)	**PA/EL**	Speed and availability first
Riak	Availability (PA)	Latency (EL)	**PA/EL**	Speed and availability first

• Real-World Analogy:

PACELC at a Restaurant **During a kitchen fire (partition):** Do you keep serving food that might be wrong (PA) or close the restaurant until the fire is out (PC)?
During normal operation (else): Do you serve food fast with possible mistakes (EL) or take extra time to ensure every dish is perfect (EC)?
Cassandra (PA/EL) = fast food that's always open. MongoDB (PC/EC) = fine dining that closes to guarantee quality.

18 NoSQL Aims

• CLASS TEST ALERT:

NoSQL Aims – Three Specific Goals The lecture lists exactly three aims. Memorise them with their one-line explanations:

1. **Flexible** – no fixed schema (data model can evolve freely)
2. **Highly available** – replication across compute clusters (always up)
3. **Web-scale** – automatic horizontal sharding (splits data across servers automatically)

19 What NoSQL Does NOT Support

• CLASS TEST ALERT:

Three Things NoSQL Drops The lecture has a specific slide titled "NoSQL do not support" listing exactly three things:

Dropped	What This Means	What Replaces It
Schema	Data model can evolve. No rigid table definitions.	Data typically exchanged as **JSON**. Dynamic schema.
SQL	No standard query language.	**Proliferation of APIs**. Simpler, lighter-weight interactions.
Transaction processing	No ACID guarantees.	**BASE consistency:** Basically Available, Soft state, Eventually consistent.

20 NoSQL Characteristics (Detailed)

• Key Concept:

Full Characteristics List from the Lecture **Distributed!**

- **Sharding:** splitting data over servers "horizontally"
- **Replication:** copying data to multiple nodes

Lower-level than RDBMS/SQL:

- Simpler ad hoc APIs
- **Programmer ensures consistency** (programming, not querying)
- Operations are simple and cheap

Different flavours (for different scenarios):

- Different **CAP emphasis** (CP vs AP)
- Different **scalability profiles**
- Different **query functionality**
- Different **data models** (key-value, document, wide-column, graph)

21 RDB vs NoSQL – Final Comparison

• Definition:

The Lecture's Summary **Relational Databases don't solve everything:**

- SQL and ACID add overhead
- Distribution not so easy

NoSQL: what if you don't need SQL or ACID?

- Something **simpler**
- Something **more scalable**
- **Trade efficiency against guarantees**

Feature	Relational (SQL)	NoSQL
Data Model	Fixed tables, rows, columns	Flexible (key-value, document, graph, column)
Schema	Predefined, rigid	Dynamic, flexible (JSON)
Query Language	SQL (standardised)	Various APIs (no standard)
Consistency	Strong (ACID)	Eventual (BASE)
Who ensures consistency?	The DBMS	The programmer
Scaling	Vertical (scale up)	Horizontal (scale out)
CAP Model	CA	CP or AP
Best For	Structured data, transactions, financial systems	Big data, high availability, web-scale
Examples	MySQL, PostgreSQL, Oracle	MongoDB, Cassandra, Redis, Neo4j

22 Practice Questions

• CLASS TEST ALERT:

Test Yourself! These questions cover ALL slides, including the previously missing content. Try without looking at notes first.

Q1. What are the three dimensions used to characterise distributed database systems? Name all levels for each.

Q2. In the taxonomy of distributed databases, what dimension values define a homogeneous system?

Q3. What is the difference between a federated loosely coupled system and a federated tightly coupled system?

Q4. Name the three requirements for large-scale internet applications from the lecture.

Q5. What mechanism do CA systems use to ensure consistency? Name the specific protocol.

Q6. During a network partition, a CP system may return no answer at all. Why?

Q7. Explain the three replication strategies from the "Critics of CAP" slide and the trade-offs for each.

Q8. What does PACELC stand for? What is Cassandra's PACELC classification and what does it mean?

Q9. Name three specific NoSQL query languages mentioned in the lecture.

Q10. What are the three aims of NoSQL? Give the one-line explanation for each.

22.1 Answers

A1. **Distribution** (D0=none, D1=client-server, D2=peer-to-peer), **Heterogeneity** (H0=same tech, H1=different tech), **Autonomy** (A0=tight/single coordinator, A1=semi-autonomous, A2=independent).

A2. **A0** (tight integration), **D2** (peer-to-peer), **H0** (homogeneous – same technology).

A3. **Loosely coupled:** Independent databases interoperate using **export schemas** (each shares only what it chooses). **Tightly coupled:** Interoperation through a **global schema** (a unified view of all databases).

A4. (1) Increasable data capacity, (2) Growing read/write throughput, (3) Handle highly concurrent access.

A5. CA systems use **replication** for availability and **transaction protocols like two-phase commit** for consistency.

A6. Because CP systems use **quorum/majority decision** algorithms (e.g. Paxos). During a partition, if consensus cannot be reached (majority can't agree), the system **delays its response** – sacrificing availability to guarantee correctness.

A7. (1) Send to all replicas simultaneously: without pre-processing → latency; with pre-processing → consistency affected. (2) Send to agreed-upon location first: synchronous → consistency; asynchronous → latency. (3) Send to arbitrary location (Dynamo, Cassandra): synchronous → consistency; asynchronous → latency.

A8. **P**artition: **A**vailability vs **C**onsistency, **E**lse: **L**atency vs **C**onsistency. Cassandra = **PA/EL** – on partition favours availability; during normal operation favours latency (speed) over consistency.

A9. **Cassandra CQL**, **Couchbase N1QL**, and Elasticsearch SQL (also: Cosmos DB, MongoDB MQL).

A10. (1) **Flexible** – no fixed schema, (2) **Highly available** – replication across compute clusters, (3) **Web-scale** – automatic horizontal sharding.

23 Quick Reference Cheat Sheet

CHEAT SHEET – Week 5 Complete

ACID (RDBMS)

- **A**tomicity – all or nothing
- **C**onsistency – always valid
- **I**solation – transactions independent
- **D**urability – changes permanent

BASE (NoSQL)

- **B**asic **A**vailability – always up (replication)
- **S**oft State – dev handles consistency
- **E**ventual Consistency – syncs later

CAP Theorem

- Pick only 2 of 3: C, A, P
- RDBMS = CA (two-phase commit)
- MongoDB/HBase/Redis = CP (Paxos)
- Cassandra/CouchDB/DynamoDB = AP
- Brewer 2000 (conjecture)
- Gilbert & Lynch 2002 (theorem)

PACELC

- On **P**artition: **A** vs **C**
- **E**lse: **L**atency vs **C**onsistency
- Cassandra = PA/EL
- MongoDB = PC/EC

NoSQL Timeline

- 1998 – Strozzi "noSQL" (was relational!)
- 2004 – Google BigTable
- 2007 – Amazon DynamoDB paper
- 2008 – Facebook open-sources Cassandra
- 2009 – Rackspace NoSQL conference

4 NoSQL Models

1. **Key-Value** – opaque values, hashtable
   Redis, Riak, DynamoDB
2. **Document** – structured JSON/XML, DB queries fields
   MongoDB, CouchDB
3. **Wide-Column** – flexible tables, sparse matrix
   Cassandra, HBase, BigTable
4. **Graph** – nodes + edges, relationships
   Neo4j, InfiniteGraph

3 Dimensions of DDBs

- **Distribution:** D0/D1/D2
- **Heterogeneity:** H0/H1
- **Autonomy:** A0/A1/A2
- Homogeneous = A0, D2, H0
- Federated = A2, Dx, Hy

3 NoSQL Aims

- **Flexible** – no fixed schema
- **Highly available** – replication
- **Web-scale** – auto horizontal sharding

3 Things NoSQL Drops

- Schema → JSON/dynamic
- SQL → APIs (CQL, N1QL, etc.)
- ACID → BASE

Replication Strategies

- Synchronous = Consistency
- Asynchronous = Low Latency
- Three approaches: all replicas / agreed location / arbitrary location

Scaling

- **Vertical (Up)** = bigger machine (RDBMS)
- **Horizontal (Out)** = more machines (NoSQL)
- Facebook: 4,000 MySQL shards (2011)